B.Tech CSE Placement Interview

Mock Interview Question Bank — SDE / Software Developer Roles

How to use this with a friend

1	Your friend picks a section and asks questions one by one. You answer **out loud**, not in your head.
2	Time yourself: 1-2 minutes for theory answers, 20-30 minutes for a coding problem.
3	Friend asks follow-ups: *Why? What if? Is there a better way?* Real interviewers always do this.
4	After every section, note: what went well, where you got stuck, what to revise.
5	Weak topics go first in the next session.

Contents

1. HR & Introduction
2. Projects & Resume
3. Programming Basics (C / C++ / Java / Python)
4. Object-Oriented Programming (OOPs)
5. Data Structures & Algorithms (Theory)
6. Coding Problems (Easy / Medium / Hard)
7. DBMS & SQL
8. Operating Systems
9. Computer Networks
10. Web Development & APIs
11. Cybersecurity Basics
12. System Design (Basic / Low-Level)
13. Aptitude & Puzzles
14. Behavioral & Situational
15. Questions to Ask the Interviewer
16. Mock Interview Scorecard

1. HR & Introduction

Time: 10-15 min | Asked in: every interview, 100%

1. Tell me about yourself.
2. Walk me through your resume.
3. Why do you want to become a software developer?
4. Why should we hire you?
5. What are your strengths and weaknesses?
6. Where do you see yourself in 5 years?
7. Why do you want to join our company? What do you know about us?
8. Why did you choose Computer Science Engineering?
9. What is your biggest achievement so far?
10. Are you comfortable with relocation, remote work, or night shifts?
11. Are you willing to sign a service agreement / bond?
12. Do you have any other offers or interviews going on?
13. What are your expectations from this job?
14. What are your hobbies and how do they help you professionally?
15. How do you keep yourself updated with new technologies?
16. Tell me something about yourself that is not on your resume.
17. Why is your CGPA low in a particular semester? (Be ready for gaps or backlogs.)
18. What will you do if you are asked to work on a technology you do not know?
19. What is more important to you: salary, learning, or work culture?
20. Do you prefer working alone or in a team?

2. Projects & Resume

Time: 15-20 min | The most grilled part. Everything on your resume is fair game.

1. Explain your best project. What problem does it solve?
2. Why did you choose this tech stack? What were the alternatives?
3. What was your exact contribution (for team projects)?
4. What was the hardest bug or challenge you faced? How did you solve it?
5. Draw and explain the architecture of your project.
6. How does the frontend communicate with the backend?
7. How did you design your database schema? Why those tables and relations?
8. How did you handle authentication and authorization?
9. How do you store passwords? Why hashing and not encryption?
10. What happens if 10,000 users use your app at the same time? What breaks first?
11. How would you scale this project?
12. What security issues could your project have? How would you fix them?
13. How did you test your project? Did you write unit tests?
14. How did you handle errors and validation?
15. Did you use Git? Explain your workflow (branches, merge, pull request).

16. If you had one more month, what feature or improvement would you add?
17. If you rebuilt this project today, what would you do differently?
18. Why did you use a REST API (or GraphQL / WebSocket) here?
19. Where is your project deployed? How did you deploy it?
20. Explain a line or function in your code that I pick randomly.
21. Which of your skills listed on the resume are you weakest in?
22. Tell me about your internship. What exactly did you work on and learn?
23. Which certification are you most proud of and what did you learn from it?

3. Programming Basics

Pick the language on your resume. Expect deep questions on it.

C / C++

1. What is the difference between C and C++?
2. What are pointers? What is a dangling pointer, null pointer, and wild pointer?
3. Difference between pass by value, pass by reference, and pass by pointer.
4. What is the difference between malloc(), calloc(), realloc(), and free()?
5. What is a memory leak? How do you avoid it?
6. Stack memory vs heap memory.
7. What are references in C++? Difference between pointer and reference.
8. What is the STL? Explain vector, map, set, and unordered_map.
9. What are smart pointers (unique_ptr, shared_ptr, weak_ptr)?
10. What is the difference between struct and class in C++?
11. What is a segmentation fault?
12. What are storage classes: auto, static, extern, register?
13. What does the const keyword do? What is a const pointer vs pointer to const?
14. Explain macros vs inline functions.
15. What is a virtual function and a virtual destructor? Why is it needed?

Java

16. Why is Java platform independent? Explain JVM, JRE, and JDK.
17. Difference between == and .equals().
18. What is the String pool? Why are Strings immutable?
19. String vs StringBuilder vs StringBuffer.
20. How does garbage collection work in Java?
21. What are checked and unchecked exceptions? Explain try, catch, finally, throw, and throws.
22. ArrayList vs LinkedList. HashMap vs Hashtable vs ConcurrentHashMap.
23. How does HashMap work internally? What happens on a collision?
24. What is the difference between final, finally, and finalize?
25. What is multithreading? How do you create a thread in Java?
26. What is the synchronized keyword? What is a deadlock?
27. What are Java 8 features: lambda, streams, Optional, functional interfaces?

28. What is the difference between an abstract class and an interface in Java?
29. What is the static keyword? Can we override a static method?
30. What is the difference between Comparable and Comparator?

Python

31. What are Python's key features? Is it compiled or interpreted?
32. List vs tuple vs set vs dictionary. Which are mutable?
33. What are decorators and generators?
34. What is the GIL (Global Interpreter Lock)?
35. Shallow copy vs deep copy.
36. What are *args and **kwargs?
37. What is a lambda function? What are list comprehensions?
38. How does memory management work in Python?
39. Difference between is and ==.
40. What are Python's mutable default argument pitfalls?

4. Object-Oriented Programming (OOPs)

Asked in almost every campus interview.

1. What is OOP? Why do we use it?
2. Explain the four pillars: Encapsulation, Abstraction, Inheritance, Polymorphism, with real-life examples.
3. Difference between class and object.
4. Abstraction vs Encapsulation.
5. Compile-time polymorphism vs run-time polymorphism.
6. Method overloading vs method overriding.
7. Abstract class vs interface. When to use which?
8. What are constructors? Types of constructors? Can a constructor be private?
9. What is a destructor?
10. What are access modifiers (public, private, protected)?
11. Types of inheritance. What is the diamond problem and how is it solved?
12. Why is multiple inheritance not supported in Java?
13. What is the difference between composition and inheritance? ('Has-a' vs 'Is-a')
14. What is a virtual function / virtual table?
15. What is early binding vs late binding?
16. What is a friend function (C++)?
17. What is the 'this' keyword?
18. What is a copy constructor? Shallow vs deep copy.
19. What are SOLID principles? Explain each briefly.
20. What are design patterns? Explain Singleton and Factory.
21. What is coupling and cohesion?
22. Can you call a constructor of the parent class from a child class? How?
23. What is the difference between a structure and a class?

5. Data Structures & Algorithms (Theory)

Time: 15-20 min | Often the warm-up before coding.

Basics & Complexity

1. What is a data structure? Linear vs non-linear. Give examples.
2. Explain Big-O, Big-Omega, and Big-Theta.
3. What is the time complexity of common operations on arrays, linked lists, stacks, queues, hash tables, and BSTs?
4. What is space complexity? What is auxiliary space?
5. Best, average, and worst case: explain with linear search and quicksort.
6. What is amortized analysis? (Dynamic array resize example.)

Arrays, Linked Lists, Stacks, Queues

7. Array vs linked list. When is each better?
8. Singly vs doubly vs circular linked list.
9. How do you detect a cycle in a linked list? Explain Floyd's algorithm.
10. How do you find the middle of a linked list in one pass?
11. Stack vs queue. Real-world uses of each.
12. How do you implement a queue using two stacks? A stack using two queues?
13. What is a circular queue? Priority queue? Deque?
14. Applications of a stack: recursion, undo, expression evaluation, parentheses matching.

Hashing

15. What is hashing? What is a hash function?
16. How are collisions handled? (Chaining vs open addressing.)
17. What is load factor and rehashing?
18. HashMap vs TreeMap vs HashSet.

Trees & Heaps

19. Binary tree vs Binary Search Tree. Properties of a BST.
20. Tree traversals: inorder, preorder, postorder, level order.
21. What is a balanced tree? Explain AVL and Red-Black tree at a high level.
22. What is a heap? Min-heap vs max-heap. How does heapify work?
23. What is a Trie? Where is it used?
24. What is the height of a tree? What is the maximum number of nodes at level k?
25. BST vs Heap: differences and use cases.
26. What is a B-tree / B+ tree and where is it used? (Databases.)

Graphs

27. How do you represent a graph? Adjacency matrix vs adjacency list.
28. BFS vs DFS: how they work, complexity, and use cases.
29. Dijkstra's algorithm. Why does it fail with negative edges? Bellman-Ford?
30. What is a minimum spanning tree? Explain Prim's and Kruskal's.

31. What is topological sorting? When is it possible?
32. How do you detect a cycle in a directed vs undirected graph?
33. What is Union-Find (Disjoint Set)?

Sorting, Searching, Techniques

34. Compare bubble, selection, insertion, merge, quick, and heap sort (time, space, stability).
35. Why is quicksort often faster than merge sort in practice?
36. What is a stable sort? Which sorts are stable?
37. Binary search: conditions, complexity, and variations.
38. Recursion vs iteration. What is tail recursion?
39. What is dynamic programming? Memoization vs tabulation.
40. Greedy vs dynamic programming vs divide and conquer.
41. What is backtracking? Example: N-Queens, Sudoku.
42. What is the two-pointer technique? Sliding window?
43. What is the 0/1 knapsack problem? Why can't greedy solve it?

6. Coding Problems

Time: 20-30 min per problem. Rule: explain your approach first, then code, then dry-run, then state the complexity.

Easy (Warm-up)

1. Reverse an array / string.
2. Check if a string is a palindrome.
3. Find the largest and second-largest element in an array.
4. Two Sum (return indices of two numbers adding to a target).
5. Check if a number is prime. Print primes up to N (Sieve of Eratosthenes).
6. Fibonacci series: recursive, iterative, and optimized (DP).
7. Factorial of a number: iterative and recursive.
8. Find duplicates in an array.
9. Check whether two strings are anagrams.
10. Valid parentheses (using a stack).
11. Count vowels / consonants / frequency of characters in a string.
12. Swap two numbers without a temporary variable.
13. Find the GCD and LCM of two numbers.
14. Check whether a number is an Armstrong number or a palindrome number.
15. Merge two sorted arrays.
16. Move all zeros to the end of an array.
17. Find the missing number in an array of 1 to N.
18. Check whether a number is a power of two.
19. Remove duplicates from a sorted array.
20. Rotate an array by K positions.

Medium (Most Frequently Asked)

21. Reverse a linked list (iterative and recursive).
22. Detect and remove a cycle in a linked list.
23. Find the middle of a linked list. Find the Nth node from the end.
24. Merge two sorted linked lists.
25. Check whether a linked list is a palindrome.
26. Maximum subarray sum (Kadane's algorithm).
27. Longest substring without repeating characters.
28. Binary search and its variations (first / last occurrence, search in rotated array).
29. Merge intervals.
30. Level order traversal of a binary tree. Height / diameter of a tree.
31. Validate a binary search tree.
32. Lowest common ancestor of two nodes in a binary tree / BST.
33. Implement a stack with getMin() in O(1).
34. Next greater element (monotonic stack).
35. Product of array except self.
36. 3Sum. Container with most water.
37. Group anagrams.
38. Number of islands (BFS / DFS on a grid).
39. Climbing stairs, house robber (basic DP).
40. Coin change problem.
41. Longest common subsequence. Longest increasing subsequence.
42. Top K frequent elements (heap / hashmap).
43. Implement an LRU cache.
44. Implement a queue using stacks.
45. Sort an array of 0s, 1s, and 2s (Dutch national flag).
46. Find the first non-repeating character in a string.
47. Detect a cycle in a directed graph. Topological sort.
48. Find the kth largest element in an array.
49. Rotate a matrix by 90 degrees. Spiral matrix traversal.
50. Check if two binary trees are identical / mirror images.

Hard (Product-Based Companies)

51. Trapping rain water.
52. Median of two sorted arrays.
53. Word ladder / shortest path in a graph (BFS).
54. Edit distance. Longest palindromic substring.
55. Serialize and deserialize a binary tree.
56. Sliding window maximum.
57. Merge K sorted lists.
58. N-Queens / Sudoku solver (backtracking).
59. Maximum path sum in a binary tree.
60. Design a data structure supporting insert, delete, and getRandom in O(1).

7. DBMS & SQL

Time: 15-20 min | Asked in almost every interview.

Concepts

1. What is a DBMS? DBMS vs RDBMS.
2. What are the advantages of a DBMS over a file system?
3. What is normalization? Explain 1NF, 2NF, 3NF, and BCNF with examples.
4. What is denormalization? When is it used?
5. Explain ACID properties with an example (bank transfer).
6. Types of keys: primary, foreign, candidate, super, unique, composite.
7. What is an index? Types of indexes (clustered vs non-clustered). When does an index hurt?
8. What is a transaction? What are COMMIT, ROLLBACK, and SAVEPOINT?
9. What are transaction isolation levels? Explain dirty read, non-repeatable read, and phantom read.
10. What is concurrency control? Locking, deadlock in DBMS, 2-phase locking.
11. What is an ER diagram? Entities, attributes, and relationships.
12. What is a view? A stored procedure? A trigger? A cursor?
13. SQL vs NoSQL. CAP theorem. When would you choose MongoDB over MySQL?
14. What is sharding and replication?
15. What is a data warehouse? OLTP vs OLAP.
16. What are functional dependencies?
17. What is the difference between a database and a schema?

SQL Queries & Commands

18. DDL vs DML vs DCL vs TCL commands.
19. DELETE vs TRUNCATE vs DROP.
20. WHERE vs HAVING.
21. Explain INNER, LEFT, RIGHT, FULL, and CROSS JOIN with examples. What is a self join?
22. UNION vs UNION ALL.
23. GROUP BY and aggregate functions (COUNT, SUM, AVG, MIN, MAX).
24. Find the second highest salary from an Employee table.
25. Find the Nth highest salary.
26. Find duplicate records in a table. How do you delete them?
27. Find employees who earn more than their managers.
28. Find departments with more than 5 employees.
29. What is a subquery? Correlated vs non-correlated subquery.
30. What are window functions? (ROW_NUMBER, RANK, DENSE_RANK.)
31. IN vs EXISTS. CHAR vs VARCHAR. NULL vs empty string.
32. What is SQL injection and how do you prevent it?
33. How would you optimize a slow SQL query?

8. Operating Systems

Time: 15 min | Key topics: processes, memory, scheduling, deadlocks.

1. What is an operating system? What are its main functions?
2. Types of OS: batch, multiprogramming, multitasking, real-time, distributed.
3. Process vs program vs thread.
4. Process states and the process control block (PCB).
5. What is a context switch? Why is it expensive?
6. User-level vs kernel-level threads.
7. Multithreading vs multiprocessing.
8. Inter-process communication (IPC) methods: pipes, message queues, shared memory, sockets.
9. CPU scheduling algorithms: FCFS, SJF, SRTF, Round Robin, Priority. Compare them.
10. What are starvation and aging?
11. What is a deadlock? Explain the four necessary conditions (Coffman conditions).
12. How can deadlocks be prevented, avoided (Banker's algorithm), detected, and recovered from?
13. Mutex vs semaphore. Binary vs counting semaphore.
14. What is a critical section? What is a race condition?
15. Classic synchronization problems: producer-consumer, readers-writers, dining philosophers.
16. What is virtual memory? What is demand paging?
17. Paging vs segmentation. What is internal vs external fragmentation?
18. What is a page fault? Page replacement algorithms: FIFO, LRU, Optimal.
19. What is thrashing? How do you handle it?
20. What is a TLB (Translation Lookaside Buffer)?
21. Stack vs heap memory in a process.
22. What is a system call? User mode vs kernel mode.
23. What is a kernel? Monolithic vs microkernel.
24. What is a zombie process? An orphan process?
25. What is the difference between fork() and exec()?
26. What is a file system? What are inodes?
27. What is the boot process of a computer?
28. Cache memory: what is it and why does it speed things up?

9. Computer Networks

Time: 15 min

1. OSI model: explain all 7 layers and their functions.
2. OSI model vs TCP/IP model.
3. What happens when you type a URL in the browser and press Enter? (Very common.)
4. TCP vs UDP. Where is each used?
5. Explain the TCP 3-way handshake and connection termination.
6. What is the difference between HTTP and HTTPS? How does TLS/SSL work at a high level?
7. HTTP methods: GET, POST, PUT, PATCH, DELETE. What is idempotency?

8. Common HTTP status codes: 200, 201, 301, 400, 401, 403, 404, 500, 503.
9. What is DNS and how does it resolve a domain name?
10. What is an IP address? IPv4 vs IPv6. Public vs private IP.
11. What is a MAC address? IP vs MAC address.
12. What is subnetting? What is a subnet mask?
13. What is NAT? What is DHCP? What is ARP?
14. Router vs switch vs hub vs bridge vs gateway.
15. What is a firewall? Types of firewalls.
16. What is a proxy server? A reverse proxy? A VPN?
17. What is a load balancer?
18. What are cookies, sessions, and tokens? What is JWT?
19. What is a CDN?
20. What are ports? Common port numbers (80, 443, 22, 21, 25, 53).
21. What is the difference between a LAN, MAN, and WAN?
22. What are sockets? What is WebSocket vs HTTP?
23. What is bandwidth vs latency vs throughput?
24. What is the difference between ping and traceroute?
25. Explain congestion control and flow control in TCP.
26. What are routing protocols? (RIP, OSPF, BGP at a high level.)
27. Error detection: parity, checksum, CRC.

10. Web Development & APIs

Ask this section if the resume lists web projects.

1. What is the difference between frontend and backend? What is full stack?
2. What is REST? What are its principles? REST vs GraphQL vs SOAP.
3. What is an API? How do you design a good REST API?
4. What is the difference between PUT and PATCH? GET and POST?
5. What is the DOM? What is event bubbling and event delegation?
6. var vs let vs const in JavaScript. What is hoisting?
7. What is a closure? What is the 'this' keyword in JavaScript?
8. What are promises and async/await? What is the event loop?
9. == vs === in JavaScript. What are arrow functions?
10. What is the difference between synchronous and asynchronous code?
11. What is the virtual DOM in React? Props vs state. What are hooks?
12. Class components vs functional components. What is useEffect?
13. What is middleware (Express / Node.js)?
14. What is MVC architecture?
15. What is CORS and why does it exist?
16. Cookies vs localStorage vs sessionStorage.
17. Authentication vs authorization. How does JWT work? OAuth at a high level.
18. What is the difference between SQL and NoSQL in web apps? What is an ORM?

19. What is caching? (Browser cache, server cache, Redis.)
20. What is responsive design? CSS Flexbox vs Grid?
21. What is Git? Explain commit, push, pull, branch, merge, rebase, and stash.
22. Merge vs rebase. How do you resolve a merge conflict?
23. What is CI/CD? What is Docker and why is it used?
24. What is the difference between monolith and microservices?

11. Cybersecurity Basics

Useful for SDE roles and essential if cybersecurity is on your resume.

1. What is the CIA triad (Confidentiality, Integrity, Availability)?
2. Hashing vs encryption vs encoding. Give examples of each.
3. Symmetric vs asymmetric encryption. Where is each used?
4. What is a digital signature? What is a digital certificate and a CA?
5. What is SQL injection? How do you prevent it? (Parameterized queries.)
6. What is XSS (Cross-Site Scripting)? Stored vs reflected vs DOM-based. How do you prevent it?
7. What is CSRF? How do you prevent it?
8. What is the OWASP Top 10? Name a few items.
9. What is a man-in-the-middle attack?
10. What is a DoS vs DDoS attack? How can it be mitigated?
11. What is phishing? Social engineering?
12. What is the difference between a virus, worm, trojan, and ransomware?
13. What is a brute-force attack? How do you protect against it (rate limiting, MFA)?
14. What is multi-factor authentication?
15. How should passwords be stored? (Hashing, salting, bcrypt / Argon2.)
16. What is penetration testing? Black-box vs white-box?
17. What is a firewall, IDS, and IPS?
18. What is digital forensics? Explain the typical process. (Identification, preservation, collection, analysis, reporting.)
19. What is the chain of custody and why does it matter?
20. What is the principle of least privilege?
21. What is a zero-day vulnerability?
22. How would you secure a login page?
23. How does this security knowledge help you as a software developer?

12. System Design (Basic / Low-Level)

Freshers get basic design questions. Focus on thinking out loud, trade-offs, and clarity.

1. Design a URL shortener (like bit.ly).
2. Design a parking lot system (low-level design with classes).
3. Design a library management system (classes and relations).
4. Design an ATM / banking system.
5. Design a simple chat application.

6. Design a to-do list / notes application with a database schema.
7. Design an elevator system.
8. Design a tic-tac-toe or chess game (class design).
9. Design a rate limiter.
10. Design a notification system.
11. How would you scale a web app from 100 to 1 million users?
12. What is caching? Where would you add caching in a system?
13. What is horizontal vs vertical scaling?
14. What is a message queue? When would you use one?
15. What is database indexing, sharding, and replication in a design context?
16. What is the CAP theorem?
17. How does consistent hashing work? (Basic idea.)

13. Aptitude & Puzzles

Common in written rounds and some HR/technical interviews.

Puzzles

1. You have 8 balls and one is heavier. Find it using a balance in the minimum number of weighings.
2. Two ropes each take 1 hour to burn (non-uniformly). Measure 45 minutes.
3. You have 3 and 5 litre jugs. Measure exactly 4 litres.
4. 100 doors puzzle: how many doors remain open after 100 passes?
5. A man has to cross a river with a wolf, a goat, and a cabbage. How?
6. How many times do the hands of a clock overlap in 12 hours?
7. Mislabelled boxes puzzle (apples, oranges, mixed): fix labels with one pick.
8. How many handshakes happen if 10 people each shake hands once with everyone else?

Aptitude Topics to Revise

- Percentages, profit and loss, simple and compound interest
- Time and work, time-speed-distance, pipes and cisterns
- Ratio and proportion, averages, mixtures
- Permutations and combinations, probability
- Number series, coding-decoding, blood relations, seating arrangement
- Data interpretation, logical reasoning, verbal ability (reading comprehension, grammar)

14. Behavioral & Situational

Use the STAR method: Situation, Task, Action, Result.

1. Tell me about a time you failed. What did you learn?
2. Tell me about a time you had a conflict in your team. How did you resolve it?
3. Describe a situation where you worked under a tight deadline.
4. Tell me about a time you showed leadership.
5. Describe a challenge you faced and how you overcame it.
6. Tell me about a time you learned something completely new quickly.

7. How do you handle criticism or negative feedback?
8. How do you prioritize when you have multiple tasks?
9. What would you do if a teammate is not contributing?
10. What would you do if you disagree with your manager's decision?
11. What if you do not know the answer to a question in the middle of a project?
12. How do you handle stress and pressure?
13. Describe a time you went beyond what was asked of you.
14. What motivates you?
15. What would you do if you found a bug in production right before a release?
16. How do you ensure the quality of your code?
17. How do you explain a technical idea to a non-technical person?
18. Are you a team player? Give an example.
19. What will you do if you do not get selected in this interview?

15. Questions to Ask the Interviewer

Almost every interview ends with 'Do you have any questions for us?' Never say no.

1. What does a typical day look like for a fresher in this role?
2. What tech stack does the team use?
3. What does the onboarding and training process look like?
4. What are the expectations from this role in the first 6 months?
5. What learning and growth opportunities does the company offer?
6. What do you enjoy most about working here?
7. What are the next steps in the hiring process?

Avoid asking: *salary and leave policies in a first technical round, or anything easily found on the company website.*

16. Mock Interview Scorecard

Friend fills this in after each mock session. Rate 1 (weak) to 5 (excellent).

Area	Score (1-5)	Strong points	Needs improvement
HR / Introduction			
Projects & Resume			
Programming Language			
OOPs			
DSA Theory			
Coding Problem Solving			
DBMS & SQL			
Operating Systems			
Computer Networks			
Web / APIs			
Cybersecurity			
System Design			
Behavioral			
Communication & Confidence			

Final tips

- Always think out loud. Interviewers hire for how you think, not just for correct answers.
- If you do not know an answer, say so honestly and explain how you would find it. Never bluff.
- For coding: clarify the problem, discuss brute force first, then optimize, then code, then test with edge cases.
- Know every line of your resume. Remove anything you cannot explain confidently.
- Revise 20-30 core questions from DBMS, OS, CN, and OOPs until you can answer them in under 90 seconds each.